NAME : SIDAK SINGH CHHABRA

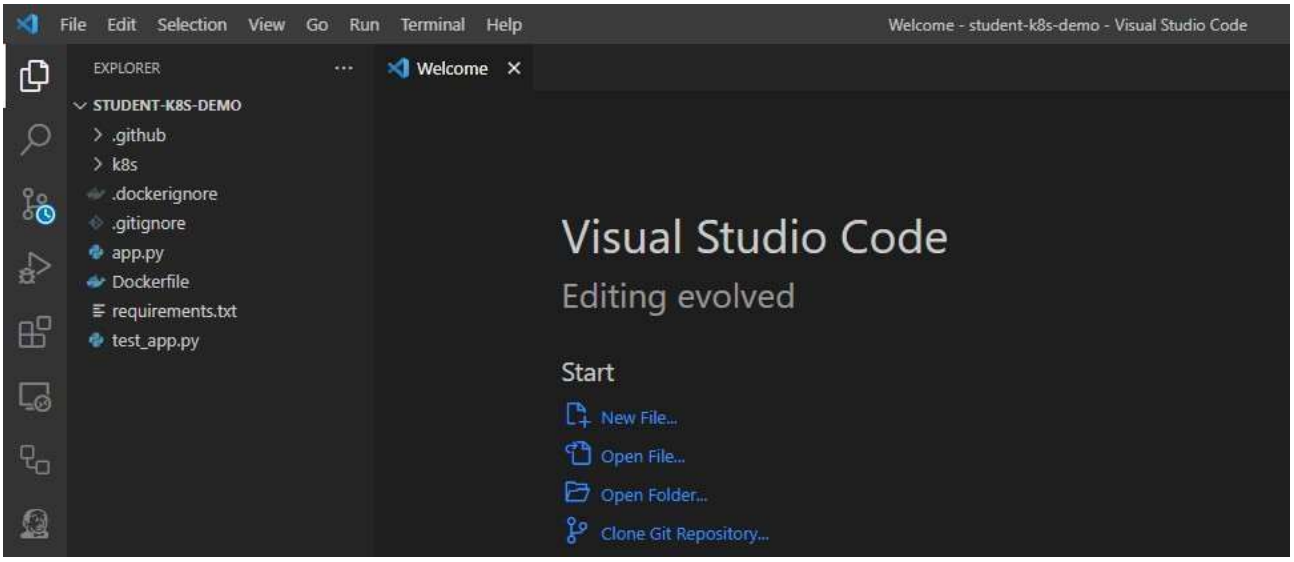
REG NO : 24MIS0414

AGILE DEVELOPMENT LAB ASSESSMENT 10

Step 1 — Create the project folder

Create a folder named student-k8s-demo and open it in VS Code. Create this structure:

```
student-k8s-demo/
├── app.py
├── requirements.txt
├── test_app.py
├── Dockerfile
├── .dockerignore
├── .gitignore
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
└── .github/
    └── workflows/
        └── pipeline.yml
```



Step 2 — Create app.py

```
from flask import Flask, jsonify

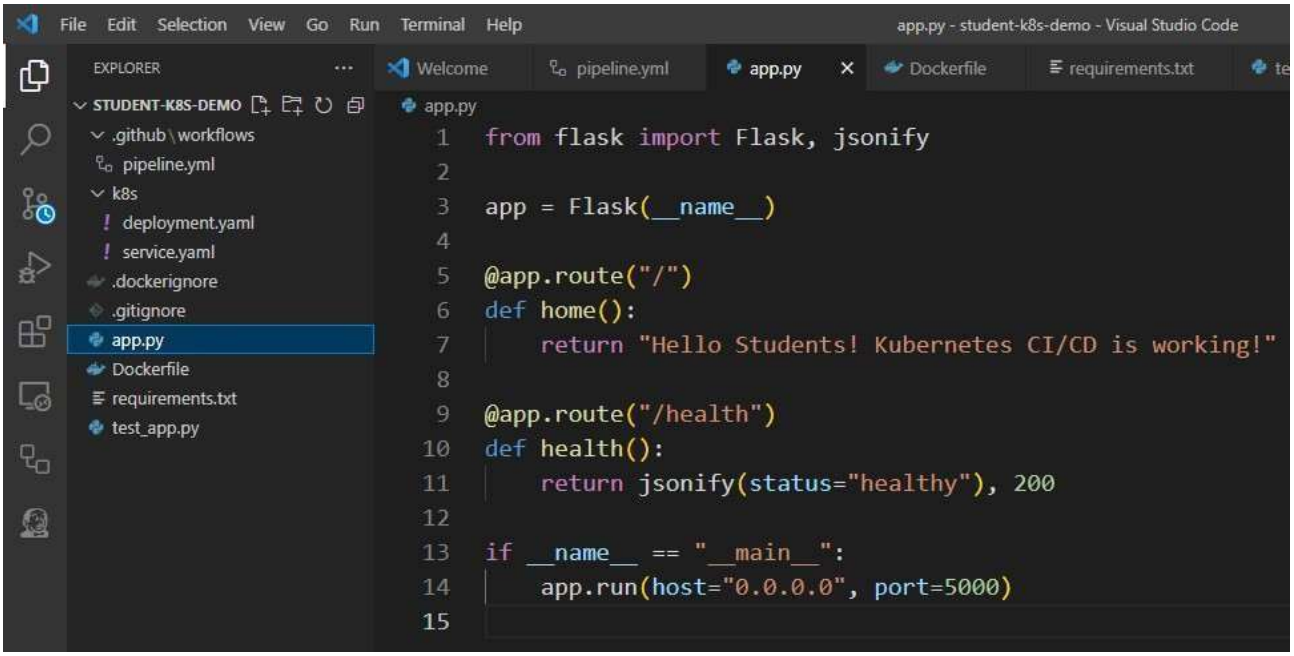
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Students! Kubernetes CI/CD is working!"

@app.route("/health")
def health():
    return jsonify(status="healthy"), 200

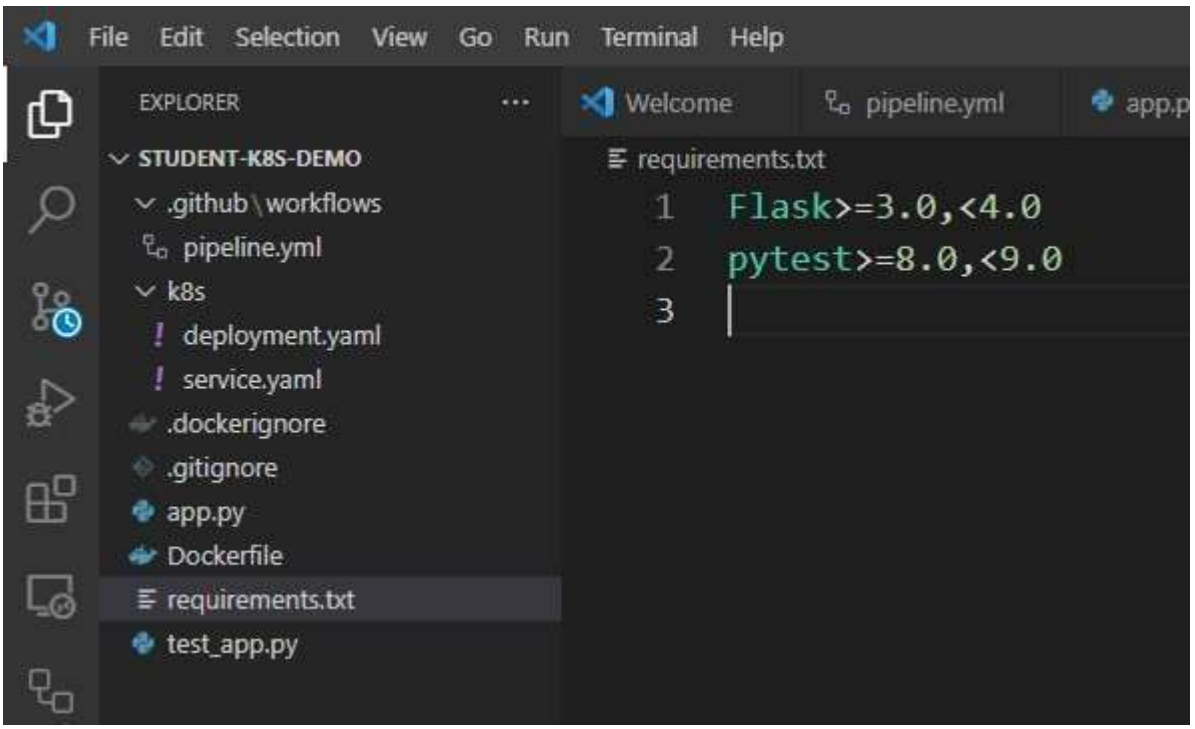
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

The home route displays a message. The /health route returns a simple health status. The application listens on port 5000.



Step 3 — Create requirements.txt

```
Flask>=3.0,<4.0
pytest>=8.0,<9.0
```



Step 5 — Run locally in Windows VS Code

Windows PowerShell may block the virtual-environment activation script with an error saying that running scripts is disabled. This does not necessarily mean the virtual environment failed to create. Use either option below from the student-k8s-demo project folder.

Option 1 — Use the virtual environment without activating it (recommended)

Run each command in the VS Code PowerShell terminal. This method does not require changing the PowerShell execution policy.

1. Install the packages:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

2. Run the automated tests:

```
.\.venv\Scripts\python.exe -m pytest -v
```

Expected result: both tests pass. If a test fails, read the error message and check app.py and test_app.py.

3. Start the Flask application:

```
.\.venv\Scripts\python.exe app.py
```

Keep this terminal open while testing. Open these addresses in a browser:

```
Home page:     http://127.0.0.1:5000/
Health check:  http://127.0.0.1:5000/health
```

The home page should show the welcome message. The health endpoint should return {"status":"healthy"}. Stop the server with Ctrl+C in the terminal.

```
PS C:\Users\batch1\student-k8s-demo> python -m venv .venv
>>
PS C:\Users\batch1\student-k8s-demo> .\.venv\Scripts\python.exe -m pip install -r requirements.txt
>>
Collecting Flask<4.0,>=3.0
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 103.4/103.4 KB 2.0 MB/s eta 0:00:00
Collecting pytest<9.0,>=8.0
  Downloading pytest-8.4.2-py3-none-any.whl (365 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 365.8/365.8 KB 7.6 MB/s eta 0:00:00
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting click>=8.1.3
  Downloading click-8.5.0-py3-none-any.whl (125 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 125.3/125.3 KB ? eta 0:00:00
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.9-py3-none-any.whl (228 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 228.7/228.7 KB ? eta 0:00:00
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.4-cp310-cp310-win_amd64.whl (14 kB)
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 134.9/134.9 KB 7.8 MB/s eta 0:00:00
Collecting pygments>=2.7.2
  Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Collecting iniconfig>=1
  Downloading iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Collecting pluggy<2,>=1.5
  Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
```

.

```
  Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Collecting iniconfig>=1
  Downloading iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Collecting pluggy<2,>=1.5
  Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Collecting colorama>=0.4
  Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Collecting exceptiongroup>=1
  Using cached exceptiongroup-1.3.1-py3-none-any.whl (16 kB)
Collecting packaging>=20
>>
============================= test session starts ==============================
platform win32 -- Python 3.10.5, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\batch1\student-k8s-demo\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\batch1\student-k8s-demo
collected 0 items

============================== no tests ran in 0.22s ===============================
PS C:\Users\batch1\student-k8s-demo> .\.venv\Scripts\python.exe app.py
>>
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.30.134.70:5000
Press CTRL+C to quit
PS C:\Users\batch1\student-k8s-demo> ^C
PS C:\Users\batch1\student-k8s-demo>
```

.

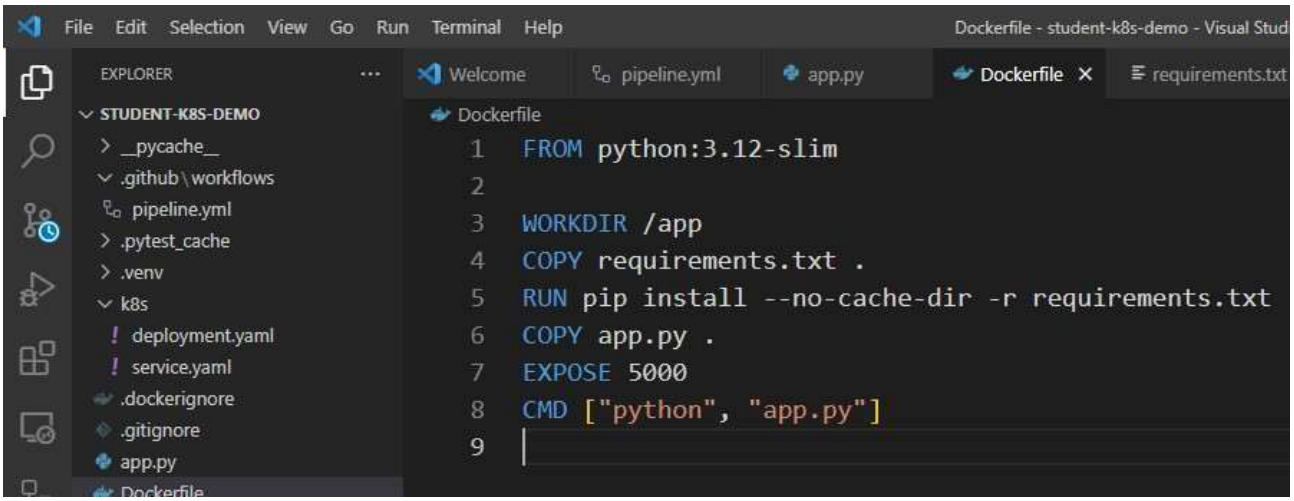
.

4. Build and run the Docker image

Step 6 — Create Dockerfile

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```



Step 7 — Create .dockerignore

```
.venv/
__pycache__/
.pytest_cache/
.git/
```

Step 8 — Build the image

```
docker build -t student-k8s-demo:v1 .
docker images
```

The dot at the end tells Docker to use the current directory as the build context.

Step 9 — Run and test the container

```
docker run --rm -d --name student-demo -p 5000:5000 student-k8s-demo:v1
docker ps
docker logs student-demo
```

Open http://localhost:5000/. Stop the container with:

```
docker stop student-demo
```

If host port 5000 is already occupied, use -p 5001:5000 and open http://localhost:5001/.

Checkpoint 2: Students can build the image, run a container, open the app, inspect logs, and stop the container.




```
PS C:\Users\HP\student-k8s-demo> docker build -t student-k8s-demo:v1 .
[+] Building 30.3s (11/11) FINISHED docker:desktop-
 => [internal] load build definition from Do  0.1s
 => => transferring dockerfile: 215B          0.1s
 => [internal] load metadata for docker.io/l  5.2s
 => [auth] library/python:pull token for reg  0.0s
 => [internal] load .dockerignore             0.1s
 => => transferring context: 135B             0.0s
 => [internal] load build context             0.1s
 => => transferring context: 418B             0.0s
 => [1/5] FROM docker.io/library/python:3.12  8.1s
 => => resolve docker.io/library/python:3.12  0.1s
 => => sha256:bbc5a12237474ec0b2 249B / 249B  0.5s
 => => sha256:6912e23eb445 12.12MB / 12.12MB  4.2s
```

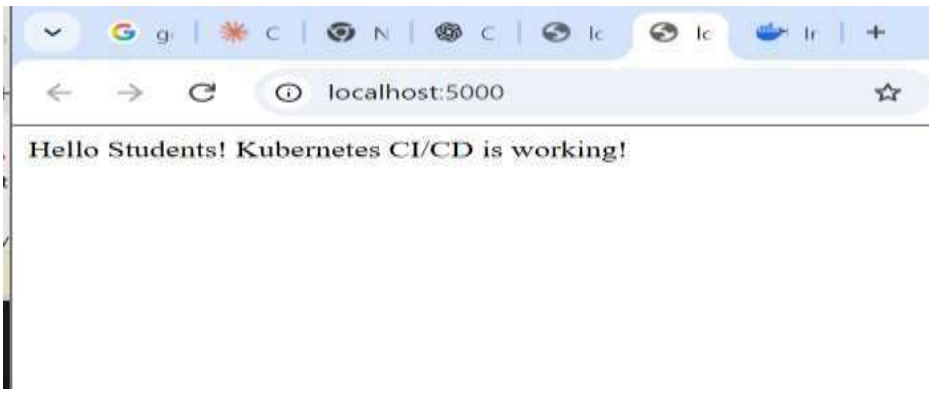
.

```
 => => exporting manifest list sha256:cc0ce3  0.0s
 => => naming to docker.io/library/student-k  0.0s
 => => unpacking to docker.io/library/studen  3.1s
PS C:\Users\HP\student-k8s-demo> docker images
                                 i Info →  U  In Use
IMAGE                ID             DISK USAGE
student-k8s-demo:v1  cc0ce3405e84       216MB
PS C:\Users\HP\student-k8s-demo> docker run --rm -d --name student-demo -p 5000:5000 student-k8s-demo:v1
a1d17005dfaf775955e41c2e44e70a7817e328bcbf6f85492eb32cc7f8e89d93
PS C:\Users\HP\student-k8s-demo> docker ps
CONTAINER ID   IMAGE                COMMAND           CREATED          STATUS          PORTS                                       NAME
a1d17005dfaf   student-k8s-demo:v1  "python app.py"   14 seconds ago   Up 12 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   stud
nt-demo
PS C:\Users\HP\student-k8s-demo> docker logs student-demo
```

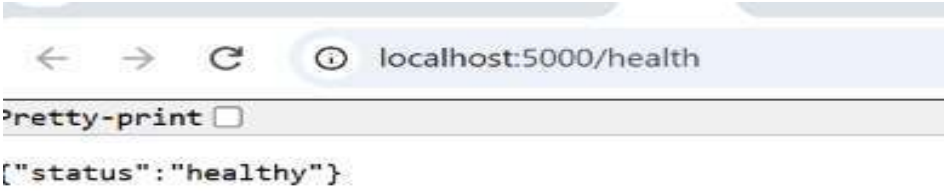
.

```
a1d17005dfaf775955e41c2e44e70a7817e328bcbf6f85492eb32cc7f8e89d93
PS C:\Users\HP\student-k8s-demo> docker ps
CONTAINER ID   IMAGE                COMMAND           CREATED          STATUS          PORTS                                       NAME
a1d17005dfaf   student-k8s-demo:v1  "python app.py"   14 seconds ago   Up 12 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   stud
nt-demo
PS C:\Users\HP\student-k8s-demo> docker logs student-demo
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
PS C:\Users\HP\student-k8s-demo>
```

.



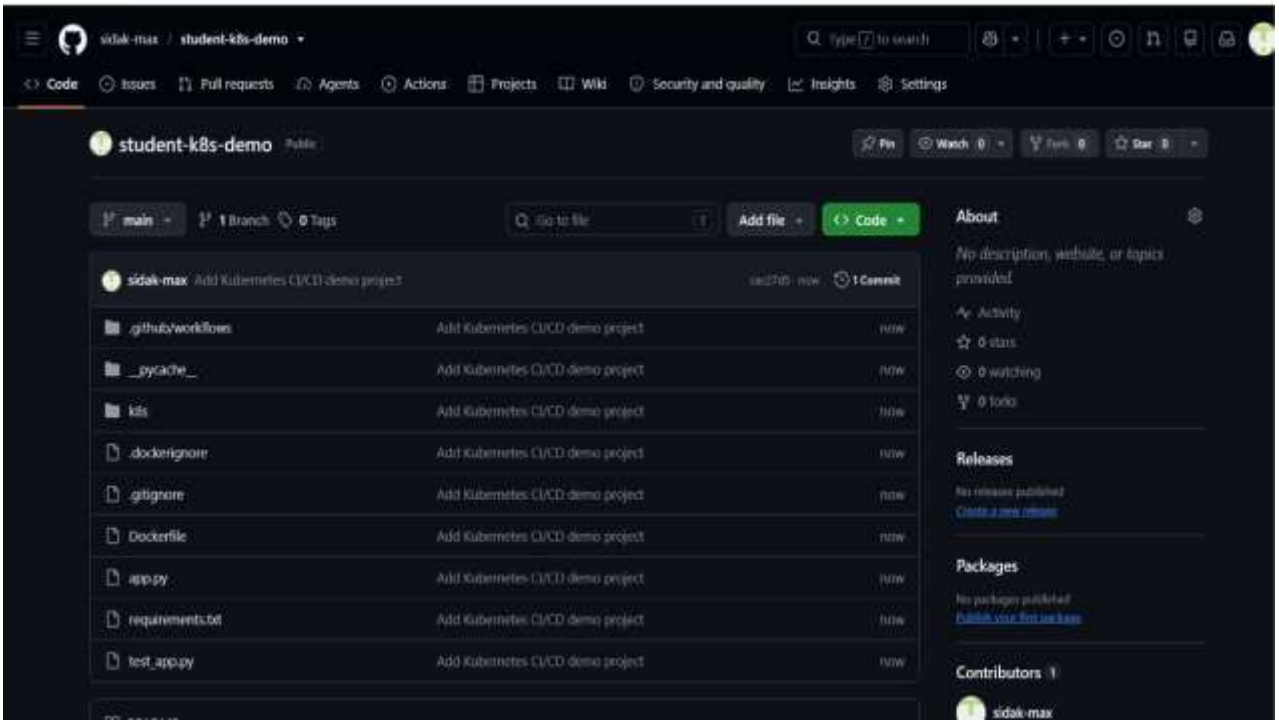
.



.

Step 10 — Create the repository

1. Sign in to GitHub and choose New repository.
2. Name it student-k8s-demo.
3. For this classroom exercise, select Public.
4. Do not initialize the remote repository with a README; the local folder already contains the project.
5. Create the repository and copy its HTTPS URL.



-

Step 11 — Add .gitignore

```
.venv/
__pycache__/
.pytest_cache/
.env
```

Then run the commands below from the project directory. Replace YOUR_USERNAME with the student's GitHub username.

```
git init
```



```
git add .
git commit -m "Initial Flask application and Docker setup"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/student-k8s-demo.git
git push -u origin main
```

If Git asks for authentication, use GitHub's supported sign-in flow. Never put passwords or access tokens in source files or commit them to the repository.

-

Step 12 — Create .github/workflows/pipeline.yml

```yaml
name: Student Kubernetes CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

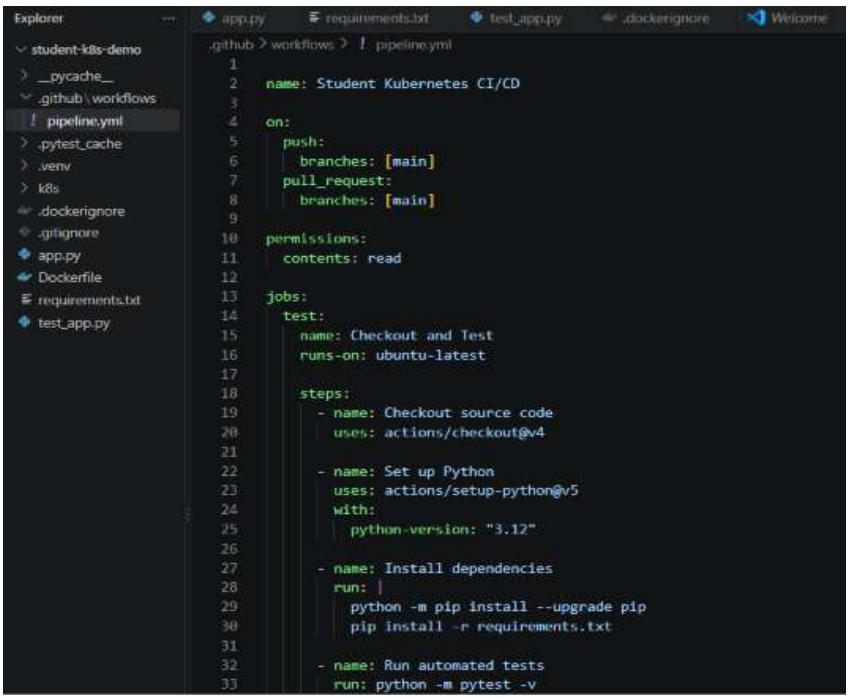
jobs:
  test:
    name: Checkout and Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

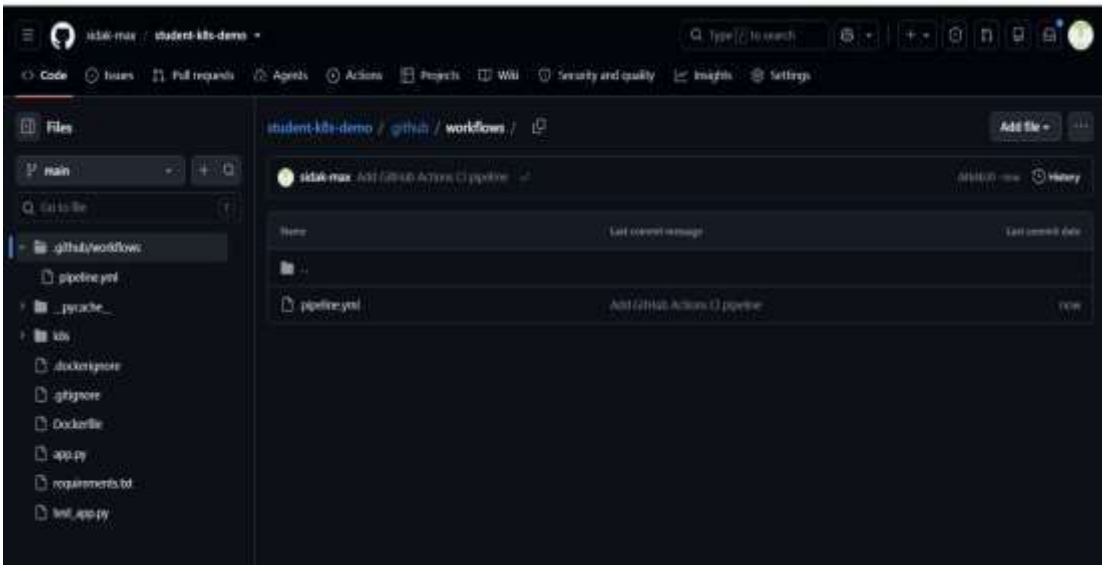
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run automated tests
        run: python -m pytest -v
```

- 

-



.

Step 13 — Commit and push the workflow

```
git add .
git commit -m "Add automated CI tests"
git push
```

.

```
PS C:\Users\HP\student-k8s-demo> git add .
PS C:\Users\HP\student-k8s-demo> git commit -m "Add automated CI tests"
On branch main
Your branch is up to date with 'origin/main'.

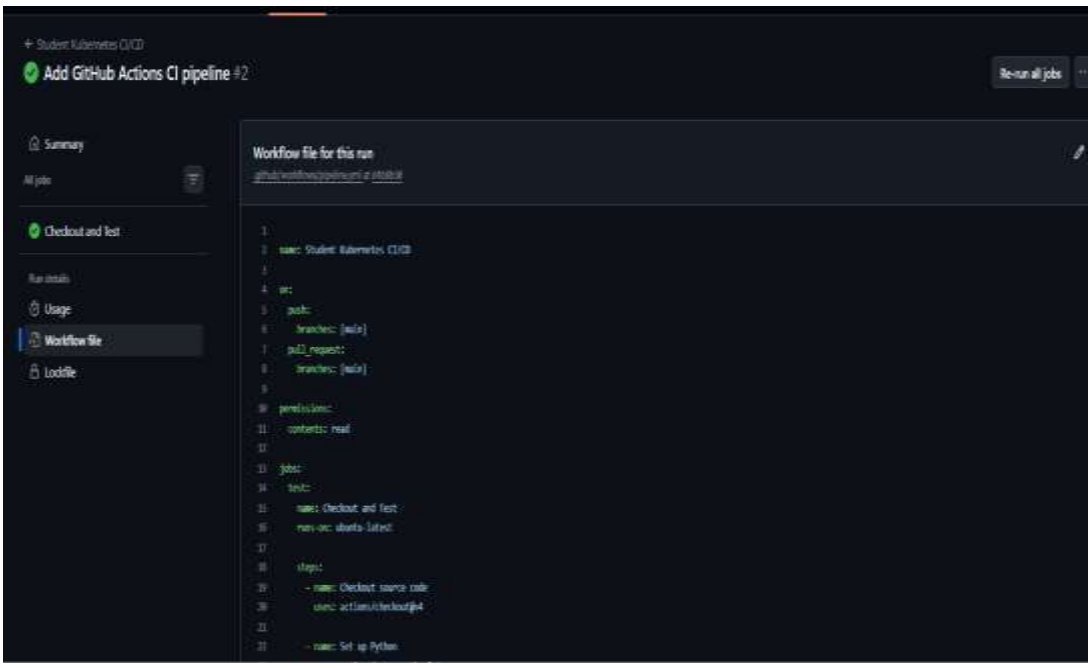
nothing to commit, working tree clean
PS C:\Users\HP\student-k8s-demo> git push
Everything up-to-date
PS C:\Users\HP\student-k8s-demo>
```

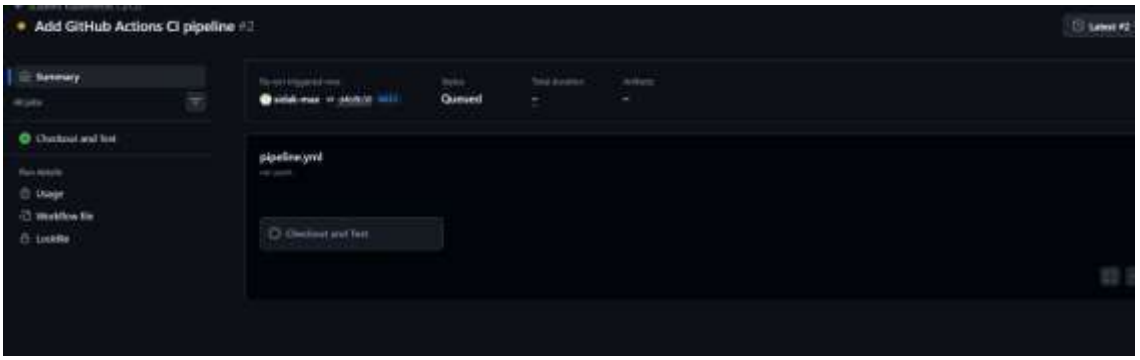
.

Step 14 — Observe the run

1. Open the repository on GitHub.
2. Click Actions.
3. Open the Student Kubernetes CI/CD workflow run.
4. Expand each step and observe checkout, Python setup, dependency installation, and tests.

.





Step 15 — Demonstrate a failed test

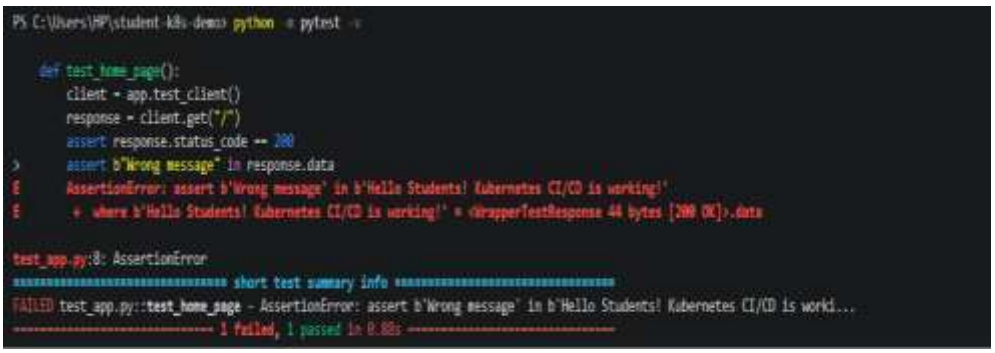
Temporarily change the expected text in test_app.py to an incorrect value:

```
assert b"Wrong message" in response.data
```



```
git add test_app.py
git commit -m "Demonstrate a failing test"
git push
```

Show that the workflow fails. Restore the original assertion, commit, and push again. The lesson: failing tests must block later publishing and deployment steps.



•



Step 16 — Create a Docker Hub repository

1. Sign in to Docker Hub and create a repository named student-k8s-demo.
2. Make it Public for this introductory lab so the local cluster can pull it without registry credentials.
3. Create an access token with the minimum permissions needed to push the image.

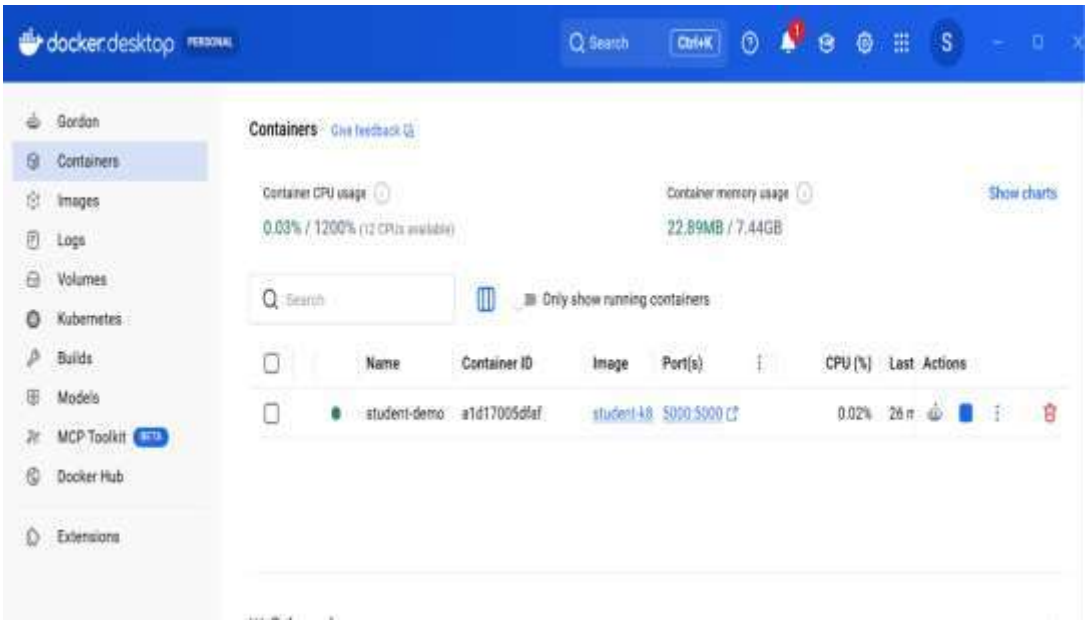
Step 17 — Add GitHub repository secrets

In GitHub, open Settings → Secrets and variables → Actions → New repository secret. Add:

Secret name	Value
DOCKERHUB_USERNAME	Docker Hub username
DOCKERHUB_TOKEN	Docker Hub access token

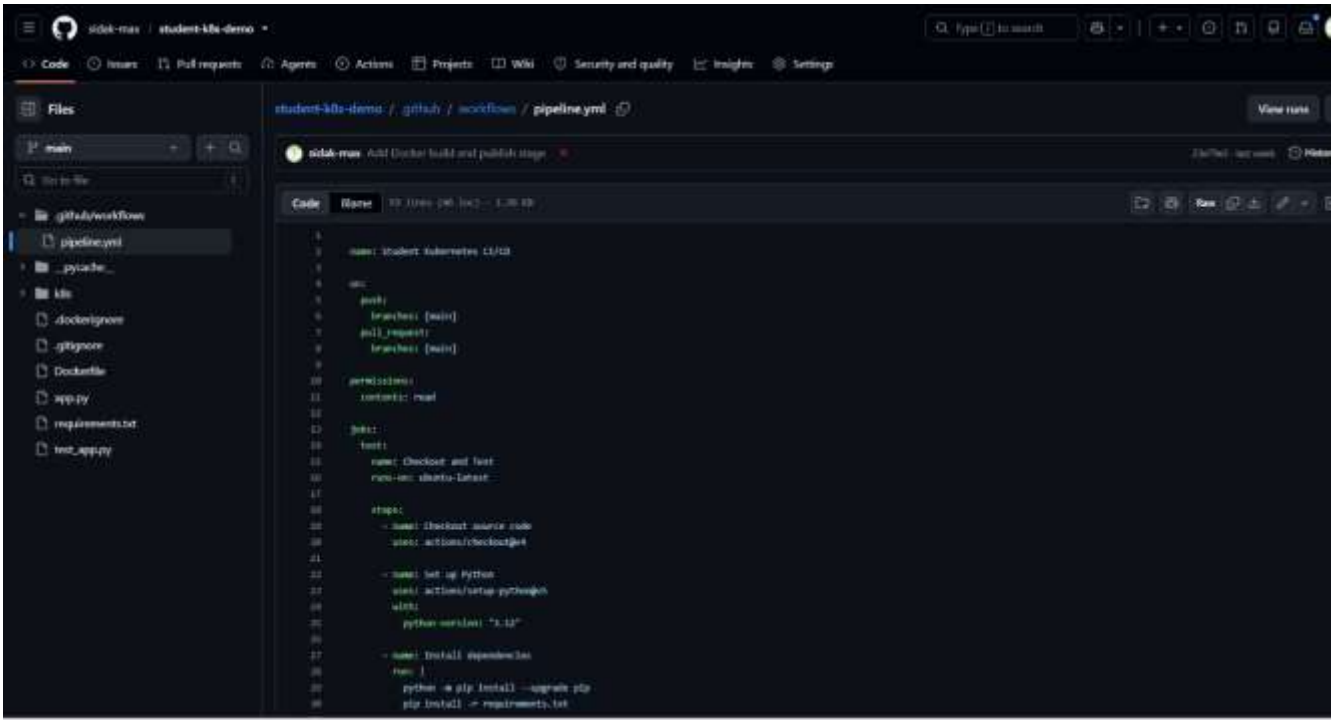
Do not hard-code credentials in YAML. Keep the image repository public for this exercise; private images need an image-pull secret in Kubernetes.

•





Step 18 — Replace pipeline.yml with this workflow

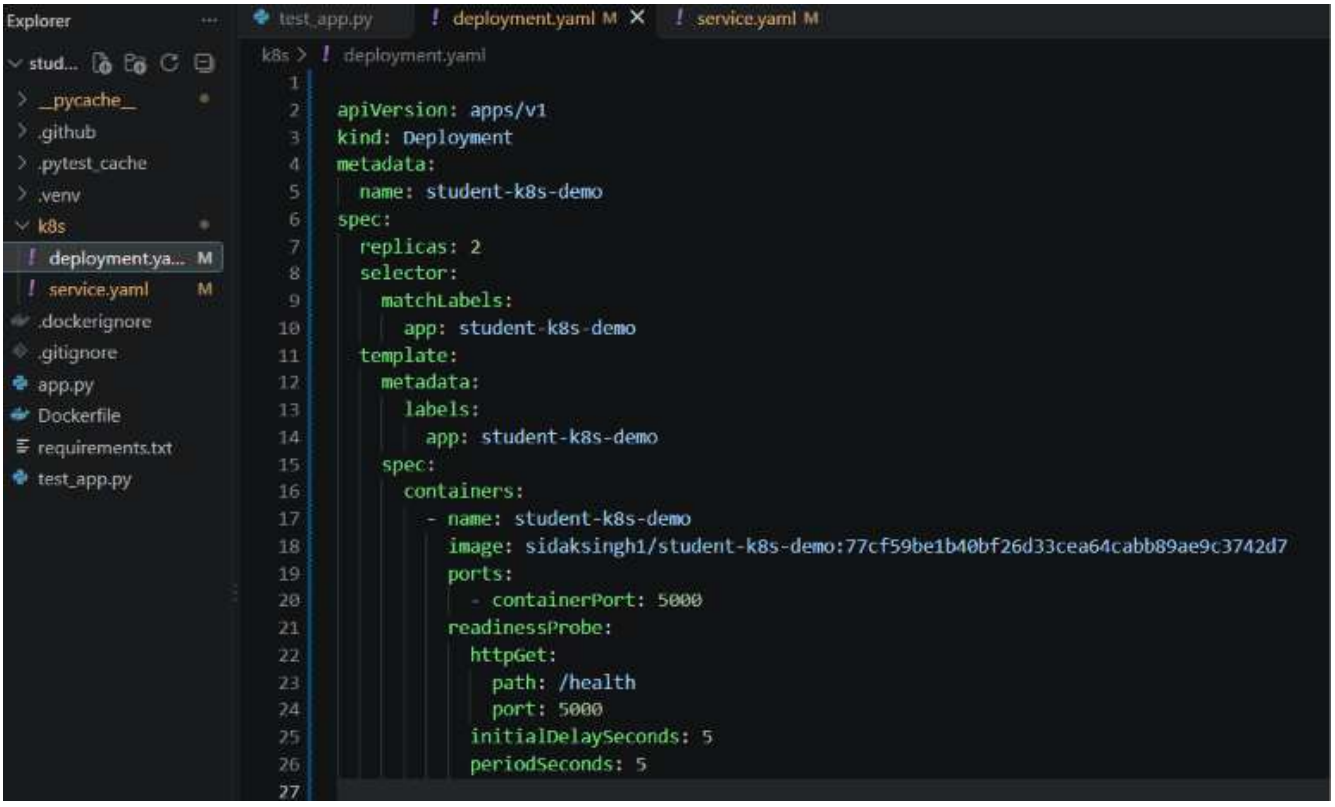


Step 19 — Create k8s/deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: student-k8s-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: student-k8s-demo
  template:
    metadata:
      labels:
        app: student-k8s-demo
    spec:
      containers:
        - name: student-k8s-demo
          image: YOUR_DOCKERHUB_USERNAME/student-k8s-demo:REPLACE_WITH_COMMIT_SHA
          ports:
            - containerPort: 5000
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 5
```

Replace the image placeholder with the actual image name and SHA shown in Docker Hub or the successful GitHub Actions run. Do not use the placeholder literally.

•



•

Step 20 — Create k8s/service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: student-k8s-demo
spec:
  type: NodePort
  selector:
    app: student-k8s-demo
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

The Service listens on port 80, forwards to the application's port 5000, and requests NodePort 30080 for local access.

